

Le debugging sous Windows



Christophe PICHAUD

.NET Rangers by Sogeti

Consultant sur les technologies Microsoft

christophepichaud@hotmail.com

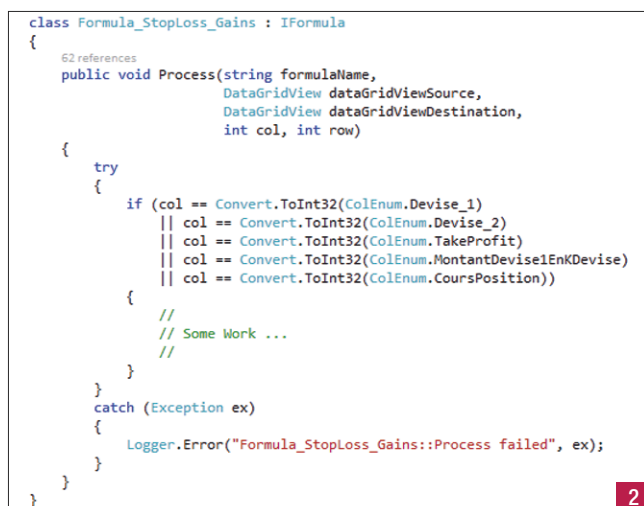
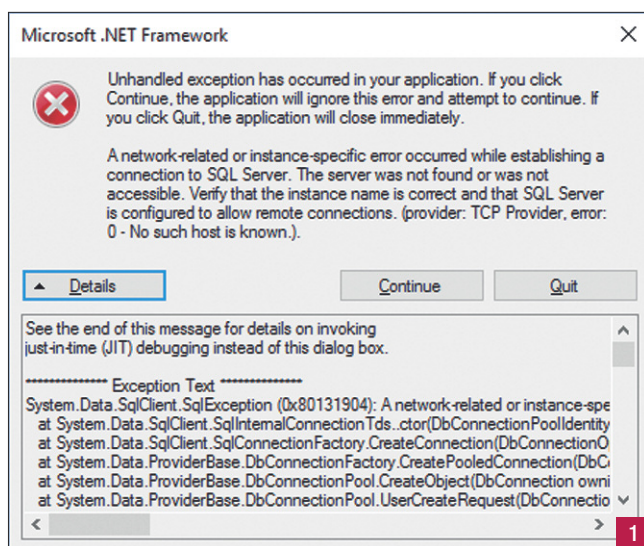
www.windowsscpp.net



Depuis que les développeurs font du logiciel, il est une pratique qui n'a cessé d'être le meilleur ami ou ennemi du développeur : le debugging. On parle du terme en anglais plutôt que du terme « mise au point ». Le debugging est un art comme une enquête policière. En effet, on cherche des pistes, on a des indices et des suspicions... On s'en relève la nuit ou on en rêve... Et quand on trouve le ou les bugs, on reçoit un flux émotionnel de soulagement. Bref, voilà la trame. Nous allons évoquer plusieurs scénarios sur du code .NET avec un outil du monde natif (C/C++) : le debugger système WinDBG.

En .NET Framework

L'avantage du développement en .NET Framework c'est que l'exception que nous cherchons a de bonnes chances d'être captée par le runtime, et une stack trace nous est proposée en appuyant sur le bouton détail. Dans mon cas, il s'agit d'une chaîne de connexion qui pointe sur un serveur que je n'ai pas... Bref vous allez me dire « c'est super simple le debugging .NET ». Oui vu comme ça pour une application GUI... [Fig.1].



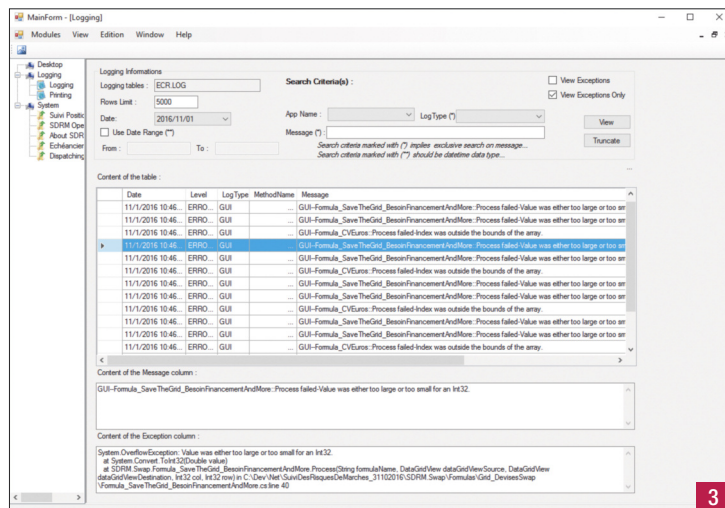
Par contre, cela peut vite devenir invivable si la boîte s'affiche toutes les 10 secondes car cela veut dire que le mécanisme de gestion des exceptions n'a pas été mis en place dans l'application.

La méthode à Papa

C'est une méthode que j'utilise depuis 20 ans et qui marche à tous les coups. Par contre c'est peut-être un bad-pattern ou un real-life pattern : faut voir... [Fig.2]. Le principe est le suivant : chaque méthode business possède au minimum un handler d'exception globale et au minimum un log est produit dans un fichier. Je suis un fan de log4net, donc dans mon cas, le log va dans la console de debug, dans des fichiers tournants (RollingFileAppender) et en base de données. Vous allez me dire, en base ? Quel est l'intérêt ? J'ai un outil de debugging qui me permet de faire des recherches dans les traces effectuées en bases, et c'est bien pratique quand des dizaines de routines business tournent en même temps. [Fig.3]. Cet outillage et cette manière de gérer les traces n'est pas valable que pour les applications graphiques. C'est aussi valable pour les services Windows ou le développement serveur.

A retenir

Il vaut mieux mettre des traces pendant le développement plutôt que de chercher aléatoirement un bout de code qui soit susceptible de planter. L'avantage du logger log4net c'est qu'il est possible de le mettre hors de service sans avoir à recompiler son code. Un simple switch dans le fichier de configuration permet d'ajuster le niveau de traces.



Comment faire sa routine de log ?

Voici un concentré de la DLL qui supporte log4net. Comme vous allez le voir c'est très simple à utiliser :

```
using log4net;
using log4net.Appender;
using log4net.Config;
using log4net.Core;
using log4net.Layout;
using log4net.Repository;
using log4net.Repository.Hierarchy;

namespace MyLibrary
{
    public class Log
    {
        private static readonly object LockingObject = new object();
        private static Log logInstance;
        private static ILog internalLog;

        private Log()
        {
        }

        public static Log Instance
        {
            get
            {
                if (logInstance == null)
                {
                    lock (LockingObject)
                    {
                        if (logInstance == null)
                        {
                            logInstance = new Log();
                        }
                    }
                }

                return logInstance;
            }
        }

        internal string LogPath { get; private set; }
        private string EnvironmentPath { get; set; }

        public bool InitializeUserLog(string environmentPath, string fileName)
        {
            try
            {
                this.EnvironmentPath = environmentPath;
                this.LogPath = this.EnvironmentPath;
                if (Directory.Exists(this.LogPath) == false)
                {
                    Directory.CreateDirectory(this.LogPath);
                }
            }
        }
    }
}
```

```
        return this.InitializeAppenders(fileName);
    }
    catch (Exception ex)
    {
        System.Diagnostics.Trace.WriteLine("Unhandled exception " + ex.Message + "\n" + ex.StackTrace);
        return false;
    }
}

public void Debug(string msg)
{
    if (internalLog.IsDebugEnabled)
    {
        this.Logger(Level.Debug, msg);
    }
}

public void Error(string msg, Exception ex = null)
{
    if (internalLog.IsErrorEnabled)
    {
        this.Logger(Level.Error, msg, ex);
    }
}

public void Info(string msg)
{
    if (internalLog.IsInfoEnabled)
    {
        this.Logger(Level.Info, msg);
    }
}

public void Warning(string msg)
{
    if (internalLog.IsWarnEnabled)
    {
        this.Logger(Level.Warn, msg);
    }
}

private bool InitializeAppenders(string fileName)
{
    // Create the RollingFileAppender
    RollingFileAppender rla = new RollingFileAppender();
    rla.Name = "MyRollingFileAppender";
    rla.File = string.Format("{0}\\{1}_{2}.log", this.LogPath, fileName, Environment.UserName);
    rla.DatePattern = "yyyyMMdd";
    rla.MaximumFileSize = "1MB";
    rla.MaxSizeRollBackups = 10;
    rla.Threshold = Level.Debug;
    PatternLayout layout = new PatternLayout();
    layout.ConversionPattern = "[%date] - %-5level - %message%newline";
    layout.ActivateOptions();
    rla.Layout = layout;
    rla.ActivateOptions();
}
```

```
// Create the ConsoleAppender
ConsoleAppender ca = new ConsoleAppender();
ca.Name = "MyConsoleAppender";
ca.Threshold = Level.Debug;
ca.Layout = new SimpleLayout();
ca.ActivateOptions();

BasicConfigurator.Configure(new IAppender[] { rla, ca });

// Set the Root logger
Hierarchy hierarchy = (Hierarchy)LogManager.GetRepository();
Logger logger = hierarchy.Root;
logger.Level = Level.Debug;

// Set internal logger
internalLog = LogManager.GetLogger(typeof(Log));

return internalLog != null;
}

private void Logger(
    Level level,
    string message,
    Exception exception = null)
{
    LoggingEvent loggingEvent = new LoggingEvent(typeof(Log), internalLog.Logger,
Repository, internalLog.Logger.Name, level, message, exception);
    internalLog.Logger.Log(loggingEvent);
}
}
```

Avec ce bout de code qui contient une classe de Log, vous pouvez l'utiliser partout !

```
public bool InitSession()
{
    try
    {
        Log.Instance.Info("InitSession...");

        //
        // Do some work ...
        //

        return true;
    }
    catch (Exception ex)
    {
        Log.Instance.Error("InitSession failed", ex);
        return false;
    }
}
```

Remarquez comment est construite cette routine. Elle renvoie un booléen. On a une information de log à l'entrée de la fonction, et une information s'il y a eu une exception. J'aime bien cette construction. Le debugging sera très facile car on a préparé le terrain... Et puis avouons-le,

cela ne coûte rien mais pourtant, personne ne le fait... C'est dommage. Au passage, ce qui est fait ici en C# peut l'être aussi en C++. La librairie log4cpp s'utilise et se configure (presque) comme log4net. L'avantage du code .NET c'est que l'on a une stack trace directement exploitable alors qu'en natif il faut provoquer soi-même la construction de la stack trace via StackWalk, une API Win32. La classe C++ qui encapsule cette possibilité native est disponible dans (accrochez-vous) le Microsoft System Journal de 1997 par Matt Pietrek et sa classe MSJExceptionHandler. Ok, on laisse les cadavres à la cave...

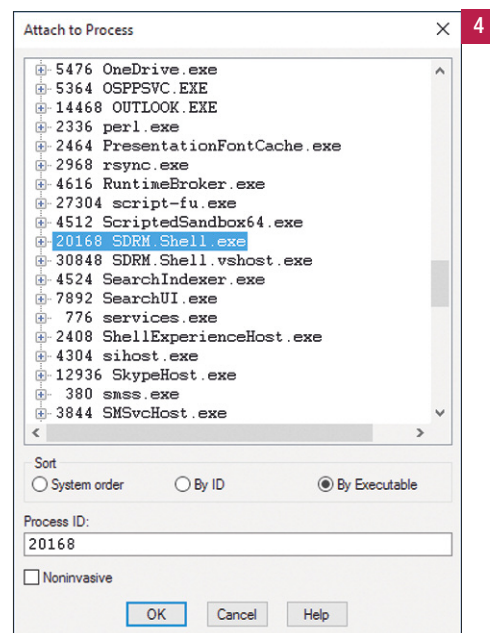
Débugger sans les sources

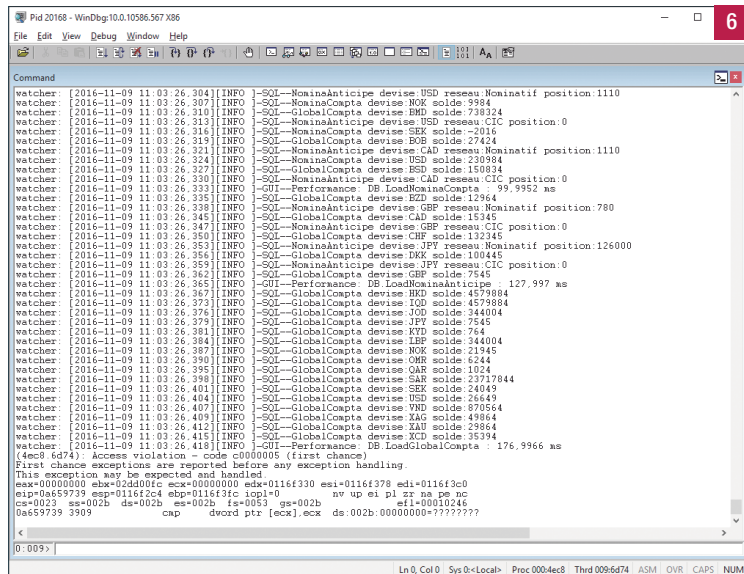
Quoi ? Vous allez me dire qu'est-ce que c'est que ce délire ??? Debugger sans les sources ??? Eh oui mon petit bonhomme... Juste avec les fichiers PDB. Vous vous souvenez que dans votre répertoire bin, il y a vos exe & dll, mais aussi des fichiers PDB. Ces fichiers sont les informations symboliques de debugging. Imaginons le scénario suivant : le serveur est mis en défaut de temps en temps et les équipes de dev n'arrivent pas à reproduire le problème. Je sens que là je vais en intéresser plus d'un ! Dans ce cas, il y a souvent un grand moment de solitude car c'est le moment où souvent, les managers rentrent dans la boucle, les emails partent, la situation se tend, et l'ambiance en prend un coup. Ce n'est pas le moment de sortir un épisode des lapins crétins ou bien une blague du commitstrip... Dans cette situation, pas la peine de chercher sur stackoverflow ; c'est à votre système d'exploitation de faire le taf. Oui votre OS Windows. On va lui demander de catcher tout seul, comme un grand, l'exception que l'on cherche, et qui n'arrive que de temps en temps... Votre ami se nomme WinDBG et c'est le debugger Windows officiel. WinDBG est distribué dans le Windows SDK ou le Windows DDK. Oui je sais, depuis que Visual Studio prend tout en charge, vous n'installez plus le SDK ou le DDK : c'est triste. Bref, téléchargez le Windows SDK et installez l'outil WinDBG.

WinDBG : le debugger système

L'avantage de WinDBG c'est qu'il sait tirer parti de suite des fichiers PDB. Vous le lancez et vous demandez à vous « attacher » à un processus. Choisissez le processus par nom ou par ID et voici ce que WinDbg affiche. [Fig.4]. Appuyez sur OK et le WinDBG va provoquer l'arrêt en breakpoint de l'application attachée. [Fig.5].

Il faut maintenant relancer l'application, car elle est à l'arrêt. Notre stratégie est simple : on laisse tourner l'application jusqu'au moment où l'on





rencontre une exception. Lancez la commande « **g** » pour go. Après plusieurs actions faites dans mon application SDRM, WinDBG stoppe l'application car un plantage (pas une exception) est survenu. Voici l'aperçu de WinDBG : [IFig.6](#). Vous remarquerez que mon logger log4net crash les logs dans le debugger. C'est très important de le faire avant « les plantages »... Cela réclame un effort qui permet d'être dans une zone de confort ensuite pour le debugginq avancé.

A ce moment-là, il faut charger « Son of Strike » alias SOS.DLL pour obtenir des informations sur le code managé car l'application est faite en C# / .NET. Lancez la commande suivante :

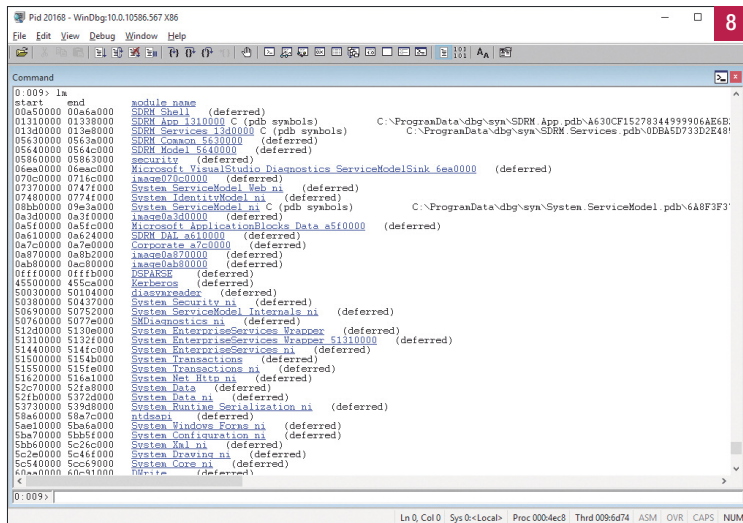
```
.load C:\Windows\Microsoft.NET\Framework\v4.0.30319\sos.dll
```

Tapez aussi **!help** et vous aurez les commandes disponibles.

Maintenant que WinDBG a arrêté le processus suite à une exception non gérée, il faut obtenir une stack des appels réalisés pour savoir d'où vient le problème. Lancez la commande suivante :

!DumpStack [Fig.7]

La callstack présente aussi bien le code du CLR (Common Language Runtime) que le code de mon application SDRM. Le debugger est très



clair : l'exception est dans la méthode *SDRM.Services.SVC.CallReutersWS-ForGrid10*. L'avantage de WinDBG est qu'il est efficace : dès qu'un plantage de type « Access Violation » survient, il met le processus en pause. Nous pouvons le relancer avec la commande « g » mais il s'arrêtera de nouveau si le plantage survient de nouveau. Vous remarquerez que le code de SDRM est avec les noms de méthodes. Vous pouvez vérifier que les symboles de debug sont chargés en exécutant le command « **lm** ».

[Fig.8]

CONCLUSION

Pour debugger sur le serveur ou sur un poste de production, il est facile de mettre des atouts de son côté pendant le développement:

- Utiliser le namespace System.Diagnostics et les méthodes Debug0 ;
- Utiliser un logger intelligent comme log4net et activer plusieurs options (console, mémoire, fichiers, db) ;
- Gérer des exceptions dans les méthodes business ;
- Installez WinDBG sur le serveur ;
- Installez ILSpy car parfois on ne se rappelle pas de tout ce que fait le code. De plus, ça peut donner des orientations sur la stratégie de debugging.

- L'utilisation de WinDBG avec sos.dll est très bien documentée sur le site MSDN. Si vous êtes curieux, vous pouvez refaire toutes les opérations que fait Visual Studio en mode debug depuis WinDBG en tapant des commandes à la main.

- programmez! - décembre 2016